USING NIKE APIS

TABLE OF CONTENTS

Using Nike APIs

Last Updated: 01/04/2023

This guide provides general information about using Nike APIs, including common standards, conventions, tips, and other helpful info that applies across multiple domains. Start here before diving into the Developer's guides.

> **TIP:** Also check out the API Basics course offered by Nike Architecture team.

Industry Standards

Learn how Nike APIs were designed with industry standards in mind.

REST Architecture

Nike uses the REST (**RE**presentational **S**tate **T**ransfer) architectural style, which allows you to communicate with Nike APIs over the Web using standard commands and protocols such as HTTP requests and responses. REST is thoroughly explained on the web already, but here are a few reasons why we use it.

Stateless for Improved Performance

REST APIs are stateless, meaning that application state is maintained on the client and not within the API itself. Each interaction with the API is done in complete isolation: your requests must always include all of the necessary information to be fulfilled by the server. In turn, the server responses must always provide all necessary information you need to create state in your app. Why is this desirable? Being stateless allows an API to be scaled across multiple servers and therefore serve millions of concurrent consumers. It also allows for easy caching, which can improve performance.

Easy Adoption = Wide Adoption

REST is widely used in the industry because the syntax and protocols used (HTTP, JSON, the URI addressing protocol) are how you already use the web. **This means less work for you to to get your app up and running**.

JSON-formatted HTTP Requests and Responses

The standard format for exchanging data with Nike APIs is JSON (**J**ava**S**cript **O**bject **N**otation). As such, all HTTP request and response payloads must be in JSON format. Wikipedia summarizes the benefits well: "JSON is a language-independent data format. It was derived from JavaScript, but as of 2017 many programming languages include code to generate and parse JSON-format data."

Example of a JSON-formatted request body that was sent to a Nike API:

```
{
    "id": "9af84d4b-6a1f-4899-af57-b4379f966913",
    "country": "US",
    "currency": "USD",
    "brand": "NIKE",
    "channel": "NIKECOM",
    "items": [
      {
        "id": "9892b8cb-e4ac-42af-a8bb-3454d8509d32",
        "skuId": "d3d15345-63e6-4cf9-9135-c2beacbbe352",
        "quantity": 2,
        "valueAddedServices": [
          {
            "id": "88d0475d-30e5-4c2f-9a87-7a31938f8ace",
            "instruction": {
              "id": "2027261230",
              "type": "customization/nike_id"
            }
          },
          {
            "id": "ac9fd9d3-3815-44ce-8b1f-18cce7abd488",
            "instruction": {
              "id": "1027551960",
              "type": "customization/nike_id"
            }
          }
        ]
      },
      {
        "id": "e945e0fd-cdcf-4005-9fe4-9302b9d6235c",
        "skuId": "f030bb22-4859-404e-82c9-aa9a23582e4f",
        "quantity": 1
      }
    ]
}
```


JSON Schema Helps Define API Contracts

The structures of the request and response bodies for Nike APIs are defined in each contract (an API.md file, commonly) using JSON Schema. Per Wikipedia: “JSON Schema specifies a JSON-based format to define the structure of JSON data for validation, documentation, and interaction control. It provides a contract for the JSON data required by a given application, and how that data can be modified.” Use the schema to understand the mandatory fields, expected data types, min/max values, and more in order to create requests and responses in accordance with the API contract. For example, here is a living example of a JSON request body schema for a Cart request.

Idempotence Guarantee

In complex distributed systems, guaranteeing that an API request will be received only once by an application can be very difficult to achieve and validate when that application is hosted across geographic regions. The idempotence guarantee states that **subsequent duplicate requests to mutate the data will not change the state of the system**. This can simplify the design when creating an eventually-consistent system, particularly when defining recovery scenarios that may need to replay API requests.

Security and Privacy

The security and privacy of your consumer’s data is Nike’s #1 concern. Whether in-flight or at rest, your consumer’s personally-identifiable information is protected according to the latest standards.

Reference Docs

In addition to the guides on the Developer Portal, all Nike APIs have the following documents available at the root directory of the GitHub repository:

- Well-defined contract (API.md or API.yaml file) for each major version in API Blueprint format. Nike has a Springboot blueprint found here.

- Request and response definitions in JSON Schema format within each API contract
- SLA.json providing the expected response time in ms nd requests per second of each service endpoint
- Consistency guarantee for mutating services within the API contract, e.g. ACID or Eventual
- README.md file including:
  - High-level explanation of the purpose of the API
  - Installation instructions
  - Configuration instructions
  - Operation instructions
  - Troubleshooting instructions
  - Email address for support, maintenance, and feature requests
  - Current version of the API (major.minor)
- CHANGES.md file listing the change history

Prerequisites

Here is what you need to do before using Nike APIs.

Registration

The Nike API registration process helps identify the software (including software version and who maintains it) that is calling an API. This is useful for understanding the impact of changes, deprecation and removal of an API.

Caller Identification Process (Optional)

JWTs are used to identify a calling service. But for APIs that do not require a JWT or requite additional information not included in the JWT, the `nike-api-caller-id` header should be passed by the calling service to identify itself to the requested service. `nike-api-caller-id` should also be passed to all downstream services.

Considerations

For this process to work between your calling app and a requested Nike API, make sure that the answer is yes to the following questions:

1. Will you always call the API through api.nike.com?
2. Will you want your app to be identified to the API with a unique caller ID that might be visible on the public internet?
3. Since the caller ID might be visible to third-parties, will you allow your caller ID to be used by a different caller?

How does it work?

1. Register a caller ID with the Product Owner of the API, with the format of `<organization_name>:<appid>:<platform>:<major>.<minor>`
2. In requests to that API, send your caller ID in request header `nike-api-caller-id`

Example header: `nike-api-caller-id: nike:dotcom:browse.wall.client:1.0`

Authorization

Consumer JWTs

This section discusses how to authorize your app or experience to call an API going through the Unified Edge Router (UER) on behalf of registered and anonymous Nike consumers. Calls to api.nike.com endpoints go through the UER and require consumer login or a visitor id for anonymous visitors.

OpenID Connect (OIDC)

Certain Nike dotcom experiences are migrating from the Unite platform for consumer login and registration to OIDC. In addition to providing standardized login and registration functionality, OIDC also offers account linking with partners and bot mitigation.

OIDC is an extension to OAuth 2.0. OIDC takes advantage of the authorization functionality OAuth provides and allows applications to receive the verifiable identity of a registered user.

For more details on OIDC, see accounts.nike.com.

OIDC or Unite?

For a complete listing of experiences that are moving to OIDC, see the NikeDotcom Unite to accounts.nike.com Mitigation Strategy.

Using the Unite Platform for Profile Management

Due to complexity issues or feasibility concerns, some Nike dotcom experiences will remain on the Unite platform. Unite offers unified profile management services such as login, registration, and session management across Nike consumer experiences. Use Unite consumer access tokens and visitor IDs to make requests on behalf of both registered Nike consumers and anonymous consumers.

Registered Nike Consumers

Those endpoints requiring consumer log in also require the experiences calling them to prove they are authorized to make API calls on behalf of consumers. After consumers log into their Nike account through the Unite platform in your app or experience, Unite returns a consumer access token (JWT) in the response. Clients pass this token in the `Authorization` header to prove they are authorized to make the API call on behalf of a registered Nike consumer. The UER does the following:

- Validates the access token
- Extracts the consumer's upmid and appid from the `Authorization` header
- Adds them as `upmid` and `appId` request headers
- Routes the request to the endpoint

Passing a consumer access token eliminates the need for clients to pass the consumer-sensitive UPMID in the API call.

Anonymous Consumers

In addition to supporting registered Nike consumers, an API may also support "Guest requests" for anonymous visitors going through the UER. Unite handles anonymous users by generating a UUID to uniquely identify the consumer visitor. Pass this UUID in the `x-nike-visitorid` header to the endpoint requiring authorization.

Table 1a: Required Headers for Consumer JWT by Consumer Type

Header	Description	Member	Guest	Employee
`Authorization`	Access token in the format of `Bearer {token}` generated by the Unite API when the consumer successfully logs in.	X		X
`x-nike-visitorid`	UUID for the guest (i.e. not logged-in) consumer, generated by the Unite API, validated by the UER, and passed through to the service		X	

Unite SDKs:

- Web SDK
- Android SDK
- iOS SDK

For more information on Unite login and JWTs, see the links below.

- Nike Consumer Login Basics
- Consumer Access Token Basics
- AAA - Getting Started with JWTs

Service-to-Service JWTs

Some services are only called by other services and require the calling service to both identify itself and prove it has access to call the endpoint. In this case, calling services need to generate S2S JWTs. The calling service signs the S2S JWT using the private key and the target endpoint uses the public key to validate the JWT. The JWT contains a list of base64-encoded “claims” that contain the calling service ID, list of services it is authorized to call with this JWT, and a list of resources/behaviors it scoped to such as ‘read,write.’

Once your service has generated the S2S JWT, send it in the `X-Nike-Authorization` request header along with the application ID (e.g. “checkouts”) authorized to call the endpoint in the `X-Nike-AppId` request header. The application ID must match the service ID of the entity that signed the JWT.

Table 1b: Required Headers for S2S JWT

Header	Description
`X-Nike-Authorization`	JWT that is a service-to-service call identifier and identifies which service is making the call
`X-Nike-AppId`	Application identifier

JWTs are configured to be reusable within a certain time period, after which any calls using that JWT will be rejected. Work with the Product Owner of the API to understand the schedule for when the JWT needs to be refreshed.

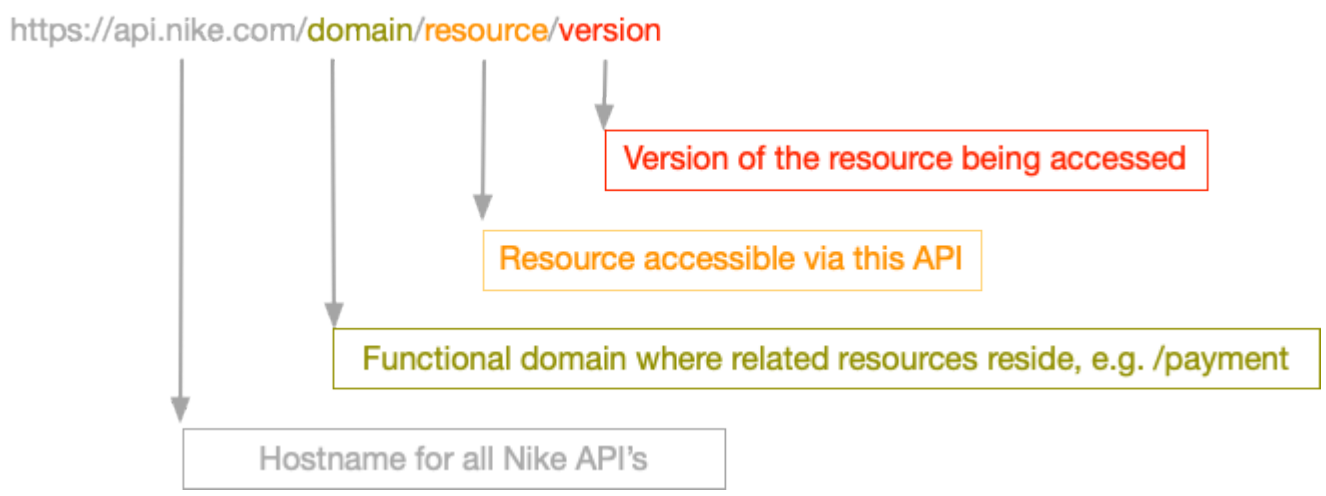
> **TIPS:**
>
> - You will need both a Production and Test JWT when calling JWT-required endpoints in those environments.
> - Service to Service (S2S) calls do not go through the UER. For endpoints that require upmid, S2S calls should send the `upmid` header for logged in consumers. Do not send the `Authorization` header.

The Nike Application Authentication and Authorization (AAA) library and the Shoestring key management tool are currently being phased out. All new applications should be using OSCAR for S2S authorization, the Chipotle library for OSCAR access token validation and the Burrito library for token generation. See these links for additional information:

- OSCAR: Server-to-server Auth Integration Guide
- FAQ: OSCAR/Chipotle
- Authentication and Authorization at Nike

URI Patterns

The standard URI pattern used for Nike APIs (v2 or later) is as follows:



For example, all Checkout APIs reside under the `/buy` domain using the URI https://api.nike.com/buy/. The resource section of the URI varies depending on the endpoint, e.g. `https://api.nike.com/buy/carts/` or `https://api.nike.com/buy/checkout_previews/`.

> **TIP:** Always check the API Developer Guide to confirm the correct URI format for a particular API.

Path Parameters

Path parameters are variable parts of a URI path. A URI can have one or more path parameters, each denoted with curly braces `{ }`. For example, in the path `/buy/carts/v1/{id}`, the `id` is the path parameter used to create or access a shopping cart resource. Nike API path parameters are always required.

Query Parameters

Query parameters can be added to the end of the URI to **allow you to be more specific about what you are requesting**.

- Query parameters in Nike APIs, with one exception (see filter table below), are in the format of `?<name>=<value>` like `?marketplace=US`.
- Multiple query parameters can be chained together with ampersands like `?marketplace=US&marketplace=EU`.
- The available query parameters vary per Nike API. Check the Developer Guide for the API in question to confirm the query parameter requirements.

> **TIP:** See also API Standards repository for more information on using query parameters with Nike APIs.

Common Query Parameters

Below is a summary of the query parameters frequently used by many Nike APIs.

Table 2: Common Query Parameters

Query Parameter	Description	Required Format	Example
`anchor`	Return only the elements after the anchor at the end of the previous page (paginated collection only)	`?anchor=<number>`	`?anchor=50`
`count`	Maximum number of elements to returned	`?count=<number>`	`?count=100`
`fields`	Restrict which fields are included (applies only to APIs with a response body)	`?fields=field1,field2,field3` or `?fields=field1.field2`	`?fields=totals(total),totals(quantity)`
`filter`	Filter results based on a provided element value	`?filter=filterName(filterValue)`	`?filter=country(US)`
`sort`	Order results by one or more fields, ascending or descending	`?sort=fieldName1Asc,fieldName2Asc`	`?sort=publishedContent.publishStartDateAsc`

> **TIPS:**
>
> - Anchor, Count, Filter, and Sort filters only apply to APIs that return a collection (a set of results, not a single result) in the response.
> - Chain more than one query parameter together using `&` in between each one, for example `?filter=channelId(008be467-6c78-4079-94f0-70e2d6cc4003)&sort=publishedContent.publishStartDateAsc,id.keywordAsc`
> - To specify nested fields, use dot notation like `?sort=fieldName.nestedFieldAsc`

Request Components

Read about the common components of HTTP requests sent to Nike APIs. Also see the related Response Components section.

URI (Universal Resource Identifier)

By sending a request to a Nike API, you access a resource using a particular URI Uniform Resource Identifier comprised of a string of characters.

> **TIP**: See the URI Patterns section for more.

HTTP Methods

Access API resources via standard HTTP methods, noting that not all APIs support all methods.

Table 3: HTTP Methods

Method	Description	Example
`GET`	Access an existing single resource or collection of resources (Idempotent)	GET /persons – List all persons
`POST`	Create a new resource	POST /persons – Add a new person
`PUT`	Update an existing resource by passing entire resource in payload (Idempotent)	PUT /persons – Bulk update persons
`DELETE`	Delete an existing resource (Idempotent)	DELETE /persons – Delete all persons
`PATCH`	Update an existing resource by passing partial updates by passing only affected fields are passed in the payload	PATCH /persons – Update one person

Request Headers

The required request headers vary per API and are described in the detailed per-API guides. This section describes the commonly-used headers by category (General, Authorization).

General Request Headers

Table 4a: Request Headers (General)

Header	Description
`accept`	Content type you will accept in response, application/json is only value allowed
`content-type`	Content type of the request, application/json is only value allowed
`true-client-ip`	IP address of the client, required if `X-Forwarded-For` is null
`x-forwarded-for`	Used to derive the IP address of the client, often required if `True-Client-IP` is null
`user-agent`	Browser and operating system of the client calling this endpoint
`upmid`	Nike profile ID of consumer, required if `x-nike-visitorid` is null
`x-nike-visitorid`	Visitor id of a non-member, required if `upmid` is null
`usertype`	'nike:swoosh' for Nike Employees, 'nike:plus' for Nike members, otherwise do not send
`origin-order-id`	Represents the legacy order id associated to the checkout and is used by downstream systems

Authorization Headers

Table 4b: Request Headers (Authorization)

Header	Description
`Authorization`	Access token
`appid`	Identifier of calling application
`X-Nike-Authorization`	JWT token
`X-Nike-AppId`	Application identifier (when JWT required)

See the Prerequisites section for more info on Authorization headers.

Request Body

Format

The default request body format is JSON (application/json) with charset UTF-8. Detailed request formats, including required and optional fields, are described in JSON Schema in each API contract (API.md file) and in the detailed per-API guides.

> **TIP:** GET requests do not require a request body.

Below are a few of the general rules, but **always follow the JSON Schema mentioned in the API.md**.

Field Names

Field names are in camelCase, for example:

- firstName
- postalCode
- startTime

Enumerations

Enumerations are in SCREAMING_SNAKE case, for example:

Table 5: Examples of Enumerations

Element	Enumeration Values
`code`	“REQUEST_INVALID”, “MISSING_REQUIRED”, “FIELD_INVALID”, “SKU_INVALID”,
`status`	“PENDING”, “IN_PROGRESS”, “COMPLETED”

Response Components

Learn about the common components of HTTP responses returned by Nike APIs. Also see the related Request Components section.

HTTP Status Codes

The HTTP protocol defines status codes to clearly describe the result of an API call. Here is a sampling of standard response codes you may see in responses:

Table 6: HTTP Status Codes

Code	Description
`200 (OK)`	Standard success response, including PUT/PATCH
`201 (Created)`	Standard success response for POST/PUT (create resource)
`202 (Accepted)`	Standard response for long-running async processing
`204 (No Content)`	Successful DELETE response
`301 (Moved Permanently)`	URI for the requested resource was moved permanently. New URI is given in the response. (different host name or even non-supported versions)
`304 (Not Modified)`	Used with ETags. Tells client response has not been modified so client can continue using cached version of the response
`400 (Bad Request)`	Request badly-formatted or not following the correct schema
`401 (Unauthorized)`	Not a valid access token
`403 (Forbidden)`	Valid access token, but access is forbidden to requested resource
`404 (Not Found)`	Resource not found

Code	Description
`406 (Not Acceptable)`	Resource format/type not supported (e.g. XML)
`409 (Conflict)`	Resource could not be updated due to other conflicting updates
`412 (Preconditioned Failed)`	Request headers did not meet the requirements
`413 (Request Entity too Large)`	Request body too large as defined by the server
`429 (Too Many Requests)`	Number of requests over a defined time interval (day or hour or minute) was exceeded by the client , rate limiting
`500 (Internal Server Error)`	Server error (details provided in the body)
`503 (Service Unavailable)`	Service is down

Response Headers

Every API response includes headers, but the headers sent will vary. Typical headers for Nike APIs are described below by category:

General Response Headers

Table 7a: Response Headers (General)

Header	Description
`cache-control`	Request and response cache instructions for browsers and shared caches (proxies, CDNs), measured in milliseconds
`content-encoding`	Type of encodings used on the message payload and in what order, for example gzip
`content-type`	Indicates the original media type of the resource, for example text/html
`content-length`	Size of the response body in bytes
`date`	Date/time response was sent
`expires`	Date/time after which the response is considered stale
`server`	Name of software used by origin server
`set-cookie`	Set when server sends cookie to user agent
`status`	Status of HTTP response
`vary`	Tells downstream proxies how to match future request headers to decide whether the cached response can be used rather than requesting a fresh one from the origin server.

CORS (Cross-Origin Resource Sharing) Headers

If your API request requires a CORS pre-flight message to be sent before the actual request, you may receive the following response headers:

Table 7b: Response Headers (CORS)

Header	Description
`access-control-allow-credentials`	Indicates whether or not the actual request can be made using credentials
`access-control-allow-headers`	Indicate which HTTP headers can be used when making the actual request
`access-control-allow-methods`	Specifies the method(s) allowed when accessing the resource

Header	Description
`access-control-allow-origin`	Specifies a URI that may access the resource
`access-control-expose-headers`	Lets a server whitelist headers that browsers are allowed to access

See the CORS section for more info.

Custom Headers

Nike APIs may use the following custom headers:

Table 7c: Response Headers (Custom)

Header	Description
`x-b3-traceid`	TraceId carried throughout all distributed systems for a request, e.g. `c2f80be958b69372`
`x-nike-appid`	Client identifier used to validate the Bearer token, for example `checkouts`
`x-nike-application`	Nike API application name, e.g. `productfeed`
`x-nike-authorization`	Authorization header with the 'Bearer' token, commonly referred to as JWT. Identifies and authorizes systems to call to this API
`x-nike-environment`	Nike environment, for example `prod`
`x-nike-version`	Nike API version number, for example `1.0.0.179`
`x-nike-visitid`	Number of visits by anonymous visitor, integer, for example `3`
`x-nike-visitorid`	ID for visitors

> **TIP:** For more info on how to use the `x-b3-traceid` header for troubleshooting, see the Troubleshooting section.

Example of actual response headers from the Product Feeds API:

```
access-control-allow-credentials:true
access-control-allow-headers:Accept,access,Authorization,Content-
Type,cloud_stack,Origin-Order-ID,x-nike-visitorid,x-nike-visitid,nike-api-caller-id,X-
B3-TraceId,X-B3-SpanId,X-B3-ParentSpanId,X-B3-SpanName,X-B3-Sampled
access-control-allow-methods:GET,POST,PUT,OPTIONS,DELETE,PATCH
access-control-allow-origin:https://www.nike.com
access-control-expose-headers:Date,WWW-Authenticate
cache-control:private, no-transform, max-age=28
content-encoding:gzip
content-length:26936
content-type:application/json; charset=UTF-8
date:Fri, 22 Sep 2017 17:36:43 GMT
expires:Fri, 22 Sep 2017 17:37:11 GMT
server:Jetty(9.2.15.v20160210)
set-
cookie:ak_bmsc=48F39E22CBDF84176C09B5C095EE587EB832586DFC6C00002B4AC559E110096D~pltyB9
LKTVZq1QrWD4glm75ctSn0vFp0P+XSvOCNcDRh7Uaizce2BiYaZ3QV4CuxZdUV0Tnsu5XPCRFBbHNV7k3gfMGL
8v70jStJpQBBAGfJlVLvkRvV8ItTGqwtMtCxHWSwZwfEqLLj+4YMqFxNgYKr5RNwNUpmDVYfe0SlEnX7ZTYEnK
gig2d8eEHtP6Qt8zwlNLuExKGFSpMbSwc3PqaN6Sk36ePSdfRBbJ6X0Y9Uc=; expires=Fri, 22 Sep 2017
19:36:43 GMT; max-age=7200; path=/; domain=.nike.com; HttpOnly
status:200
vary:Accept-Encoding
x-b3-traceid:c2f80be958b69372
x-nike-application:productfeed
x-nike-environment:prod
x-nike-version:1.0.0.179
```

Response Body

Your responses will usually include a body but there are certain HTTP methods that don't require response bodies to be sent. The default response body format is JSON (application/json) with charset UTF-8. Detailed response formats are described in JSON Schema in each API contract (API.md file) and in the detailed per-API guides.

Error and Warning Messages

Although Nike uses a standard formatting for error and warning messages included in a response body, the content of each is specific to an API endpoint. See the per-API guides in each endpoint section for detailed error/warning information.

For additional general error handling info, see the Error Handling section.

Using the API Reference

This section describes how to use the Nike API Reference documents on the Developer Portal.

Overview

Each Nike API Reference document serves as the contract for using the API and has the following features at a minimum:

1. Describes all available endpoints
2. For each endpoint, lists which URI query and path parameters are required or optional, including data types and sample data
3. For each endpoint, lists which types of requests and responses are allowed or expected, including headers and payload details with sample data and JSON schema for each

> **TIP**: In some cases, depending on the API, the doc will also provide details for authorization/authentication, error code information, or an overview of the API.

Accessing the API Reference on the Developer Portal

The steps for accessing the API Reference on the Developer Portal are as follows:

1. Navigate to the Developer Portal
2. Use the Search bar to locate the API and click the API name
3. Click the **API** tab to display the API Reference document

4. Scroll to the endpoint you are interested in
5. To expand a section, click **SHOW** on the line of the section you wish to display
6. Each request/response section may have a **Headers**, **Body**, **Schema** subsection
7. Scroll to the subsection of interest to display it
8. Within a **Schema** subsection, any required fields are either listed in a "Required" section or indicated at the field level

> **TIPS**:
>
> - For more on JSON Schema, see the JSON Schema Helps Define API Contracts section of this doc.
> - For more on how to use the Developer Portal, see the Developer Portal User Guide.

Making Your First Request

For your first API request, send a request to the Create or Update a Cart by Cart ID endpoint of the Carts API to create your first cart.

1. Gather Data Needed For The Request

Our example endpoint, `Create or Update a Cart by Cart ID`, supports the HTTP PUT method. To create a cart, send a PUT request with (at minimum) the required request headers and the required parts of the request body.

First, read the API Reference to learn more about the required parts of this request. Assume that the consumer for whom you are creating the cart is a Nike member who has logged in. This determines which request headers are required for this particular request.

For the request headers, the following considerations apply (at minimum):

- Always send `application/json` in both the `Accept` and `Content-Type` headers.
- Send the consumer's access token as obtained from accounts.nike.com or Nike Unite/Identity in the `Authorization` header.

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer {your access token}
```

For the request body, the following considerations apply (at minimum):

1. Send a UUID **that you have created** in the `id` field, in this case `61bc115b-16e5-43b5-bcaf-dd6168c543f8`. This is the cart ID.
2. Send the country code of the country where the consumer is shopping in the `country` field, in this case, `US`. Send `NIKE` in the `brand` field.
3. Send a UUID **that you have created, different from above** in the items.`id` field. This is the identifier for the line item in the cart. If you include multiple line items, each must have a unique identifier.
4. Send a valid SKU identifier (obtained from the Product Feeds API) in the items.`skuId` field.

```
{
  "id": "61bc115b-16e5-43b5-bcaf-dd6168c543f8",
  "country": "US",
  "brand": "NIKE",
  "items": [
    {
      "id": "9992b8cb-e4ac-42af-a8bb-3454d8509d32",
      "skuId": "1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
      "quantity": 1
    }
  ]
}
```

2. Create the URI

The API Reference for the `Create or Update a Cart by Cart ID` endpint states that the required URI format is `/buy/carts/v2/{id}`. To build the full URI, prepend `https://api.nike.com` to the above path and append `id` after "v2". The `id` is the cart identifier you passed in the request body. The complete URI is then https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8.

3. Execute the request

Execute the request with a cURL command. Using the values gathered in steps 1 and 2, the complete cURL command is:

```
curl -X PUT \
  https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8 \
  -H 'Accept: application/json' \
  -H 'Authorization: Bearer {your access token}' \
  -H 'Cache-Control: no-cache' \
  -H 'Content-Type: application/json' \
  -d '{
     "id": "61bc115b-16e5-43b5-bcaf-dd6168c543f8",
     "country": "US",
     "brand": "NIKE",
     "items": [
       {
         "id": "9992b8cb-e4ac-42af-a8bb-3454d8509d32",
         "skuId": "1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
         "quantity": 1
       }
     ]
}'
```

4. Parse the Response

Assuming no errors, you will receive a response body similar to the following:

```
{
    "id": "61bc115b-16e5-43b5-bcaf-dd6168c543f8",
    "country": "US",
    "currency": "USD",
    "brand": "NIKE",
    "totals": {
        "subtotal": 130,
        "discountTotal": 0,
        "valueAddedServicesTotal": 0,
        "total": 130,
        "quantity": 1
    },
    "items": [
        {
            "id": "9992b8cb-e4ac-42af-a8bb-3454d8509d32",
            "skuId": "1f6b36bd-61c0-5eb5-b9f0-8643e1ebae37",
            "quantity": 1,
            "priceInfo": {
                "price": 130,
                "subtotal": 130,
                "discount": 0,
                "valueAddedServices": 0,
                "total": 130,
                "priceSnapshotId": "d5d8dbed-afba-4677-9904-e9712fa3ecb9",
                "msrp": 130,
                "fullPrice": 130
            }
        }
    ],
    "links": {
        "self": {
            "ref": "/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8"
        }
    },
    "resourceType": "cart"
}
```

Some values in the response are exactly what you sent in the request, but the values in the `totals` section provide a cart pricing summary and the values in items.`priceInfo` section provide the current product pricing details.

Another Example

For your next request, send a GET request to the Get a Cart for a Cart ID endpoint of the Carts API, using the cart `id` that you created in the previous step.

For the request headers, use the same headers you used in the previous step.

> **NOTE**: There is no request body needed for a GET request.

The complete URI is `https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8`. Note that the same cart `id` that you created for the previous PUT request is at the end of the URI. The final cURL is:

```
curl -X GET \
  https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8 \
  -H 'accept: application/json' \
  -H 'authorization: Bearer {your access token}' \
  -H 'cache-control: no-cache' \
  -H 'content-type: application/json'
```

The response body from the Get a Cart for a Cart ID endpoint is the same as Create or Update a Cart by Cart ID, so the process of parsing it is also the same.

Versioning

As Nike APIs are enhanced over time to add new features and fix bugs, the version numbers are incremented according to Semantic Versioning guidelines. Some high-level considerations:

- For minor version increments or patches, e.g. the addition of a new, optional field, the changes are non-breaking and the endpoint URI does not change. If you are using the Tolerant Reader Pattern, you can continue to use the API without having to make changes to your app.
- When moving to a new, major version of an API (e.g. a change from synchronous to asynchronous operation), expect to make some changes to your app to ensure compatibility with the new version.

> **TIP:** For more details about the versioning of Nike APIs, see the API Versioning Strategy document.

Caching

Nike APIs take advantage of three layers of caching in order to keep service performance optimal even in periods of high request volume. The caching layers are:

1. Akamai Content Delivery Network (CDN)
2. Service
3. Device/Browser

Akamai Caching

Nike uses the Akamai Content Delivery Framework as the UER caching solution for public service requests. It is utilized when the client makes a request for a Nike public resource configured to go through Akamai's Edge server.

Akamai caching and routing is managed though a set of configurations at Akamai. It is referred to as an Edge server because it is on the Edge of two networks, in this case the public internet and Nike's UER. Akamai operates on a set of configured rules that determine what resources can be cached, how long to cache the resource, and how to determine if the origin of the resource has an updated version (stale resource). Akamai retrieves a cached copy of the data that is as close to the caller as possible to ensure the quickest response time.

Listed below are the Production domains that are routed to Akamai's Edge caching server:

Table 8a: Nike Domains Routed to Akamai

Domain	Description
www.nike.com	For unauthenticated, public experiences, services and assets

Domain	Description
api.nike.com	For authenticated public experiences

> **TIP**: Akamai caching is bypassed in application-to-application calls because those requests do not go through Akamai.

Service Caching

Nike Architecture encourages caching at the individual service level and discourages implementing custom distributed-caching solutions, due to high development costs. Not all Nike Cloud services support caching and cache times vary across services. Check the Caching section of the Nike API Documentation you are interested in for cache information by service.

Device/Browser Caching

Caching can also be done on the Browser/Phone device itself. This type of caching is controlled by exchanging a series of request and response headers as is demonstrated in the flow below.

1. The client sends a service request for a URI resource.
2. The service returns the resource response with an `ETag` header representing the version of the data, an `Expires` header representing how long until the URI expires, and a `Cache-Control` or `Pragma` header indicating that the response can be cached.
3. The client caches the response and `ETag` value.
4. If the client needs the same resource again, the client uses the `Expires` and `Pragma`/`Cache-Control` headers to check if the local URI cache has expired.
5. If the client cache has not expired, the client retrieves the local version from cache.
6. If the client cache has expired, the client sends a service request for the previously requested URI resource, sending the `ETag` header and `If-None-Match` header set to the cached ETag value.
7. Because the `If-None-Match` header is present, the Service checks its `ETag` value with the `ETag` value passed in the request header to check if it has a newer version of the resource.
8. If the `ETag` values match, the service returns a shortened response and with an HTTP 304 Not Modified status, greatly reducing network bandwidth.
9. If the `ETag` value doesn't match, the service returns the full response and an updated value in the `ETag` header and the process restarts.

Cache-Related Headers

Request

Table 8b: Cache-Related Headers (Request)

Header	Description	Example
`Cache-Control`	Directive indicating what and how resource can be cached	
`If-None-Match`	Used on request with ETag (sent prior by server) to only get resource if newer than tag or 304 otherwise	If-None-Match: "686897696a7c876b7e"
`If-Match`	Used with the value from the received ETag header for PUT/ PATCH to achieve optimistic locking (conditional update)	
`ETag`	String that identifies a specific "version" of the requested, mutable resource.	ETag: "686897696a7c876b7e"

Header	Description	Example
`Pragma`	Used to disable caching for a resource or at least let the client know this resource must not be cached	Pragma: “no-cache”
`Expires`	Date indicating when this resource is stale. Do not use the Pragma with this header	Expires: Fri, 01 Dec 2017 16:00:00 GMT

Response

Table 8c: Cache-Related Headers (Response)

Header	Description	Example
`ETag`	String that identifies a specific “version” of the mutable resource.	ETag: “686897696a7c876b7e”
`Expires`	Sate indicating when this resource is stale. used with `Cache-Control`	Expires: Fri, 01 Dec 2017 16:00:00 GMT
`Cache-Control`	Used for enabling caching of resources and identifying what can be cached. used with Expires.	Cache-Control: private,max-age=0
`Pragma`	Used to disable caching for a resource or let the client know this resource must not be cached. use `Cache-Control` instead.	Pragma: “no-cache”

> **TIP:** See Nike Caching Strategy and Akamai Technical Reference for more info.

CORS

Client-side HTTP requests are subject to the same-origin policy, meaning the requested resource must be for the same domain, port, and protocol as the originator of the request. This restriction prevents unsafe requests that could compromise data integrity. For instance, a request from a script on api.nike.com for an image on images.nike.com violates the same origin policy and results in a security error. One way to relax this restriction is to make CORS requests. CORS (Cross-origin resource sharing) gets around this restriction through an exchange of headers between the browser making the request and the server of the requested resource. The browser sends an Origin header containing the origin making the request (e.g. http://api.nike.com) and the server sends a Access-Control-Allow-Origin header in the response that lists all origins allowed to access it. If the origin is in the list, the browser lets the request through. For a detailed explanation of CORS and implementation examples see HTTP Access Control CORS.

Implementation Recommendations

Developers implementing Cloud Services should keep the following best practices in mind:

- Be consistent and as strict as possible in allowed CORS header values.
- Avoid using the wildcard character * as Access-Control-Allow-Origin response header value. Use a list of specific values instead.
- Control restriction of Access-Control-Allow-Headers, Access-Control-Expose-Headers, Access-Control-Allow-Methods, and Access-Control-Allow-Origin headers for both requests and responses at the service level, perhaps through the use of a Servlet Filter.
- Avoid setting Allow-Control-Allow-Credentials to true unless the service requires cookies. This value is false by default.

Asynchronous Operation

A **synchronous** endpoint returns the result immediately because both the request and work performed execute in the same thread. Synchronous endpoints are quick-running. An **asynchronous** endpoint takes a work request and promises to return the results in an estimated time in the future. It immediately returns a status, an eta, and a result link at which the caller should poll for job results. It is best practice to wait the duration of the eta before asking for job results, also known as Planned Polling. If the job still is not finished, the caller should wait for the length returned in the job result eta before polling the job result again. It is suggested that services running longer than 250ms be an asynchronous service; it is mandated that services running longer than 500ms be an asynchronous service.

Asynchronous jobs are either in PENDING, IN_PROGRESS, or COMPLETED status. PENDING indicates that the job has been accepted into the work queue. IN_PROGRESS indicates the job has been picked up from the work queue and is being actively worked on. COMPLETED indicates the job is complete and results are available. There are two or three endpoints involved in an asynchronous service: job request, job status and job result. Job status and job result endpoints are sometimes combined.

Job Request

This endpoint returns a unique job id in UUID format, a status, eta of when the job will be finished and a `self` link to the job result to poll for job status. Notice that the `self` link contains the same UUID as the job id. In this case, the caller should wait 3000ms before polling the job result link.

```
{
        "id": "5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "PENDING",
        "eta": 3000,
        "resourceType": "job",
        "links": {
                "self": {
                        "ref": "/buy/checkouts/v2/jobs/
5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        }
}
```

Job Status

This endpoint is called by passing the `id` returned form the Job Request response above. Because the job status is not `COMPLETED`, the calling service should wait 500ms before polling the job status again.

```
{
        "id": "5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "IN_PROGRESS",
        "eta": 500,
        "links": {
                "self": {
                        "ref": "/buy/checkouts/v2/jobs/
5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        },
        "resourceType": "job"
}
```

Job Result

The job is now `COMPLETED` and resulted in an error. The errors array lists details about each error including the error message and error code.

```
{
        "id": "5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "COMPLETED",
        "error": {
                "message": "Authorization failed for order C00013575031",
                "httpStatus": 409,
                "code": "PAYMENT_INVALID",
                "errors": [{
                        "message": "Payment failed",
                        "code": "PAYMENT_INVALID"
                }]
        },
        "links": {
                "self": {
                        "ref": "/buy/checkouts/v2/jobs/
5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        },
        "resourceType": "job"
}
```


Error Handling

Use this guide to understand what to expect from Nike API error responses and learn some tips on how to handle them.

General Error Response Components

Error responses from Nike APIs contain the following:

- HTTP Status Code

  Nike uses standard HTTP status codes for errors. See the Common Errors section for more details.

- Response Body Components

Table 9a: Error-Related Response Body Components

Element Name	Description
`message`	Plain text description of the error at top level of response
`errors`	Array at top level of response, containing the following:
`code`	Code for the error, text-based used ‘screaming snake case’, e.g. ‘FIELD_INVALID’
`message`	Plain text description of the error
`field`	Field in the request which had the error, in JSON Pointer format

- Response Header Components

Nike APIs return a Trace ID in the `X-B3-TraceId` response header. This can be used to query logs to troubleshoot the error.

> **TIPS:**
>
> - For more about how to use Trace IDs, see the Troubleshooting section.
> - Not all HTTP responses include a body, e.g. 204 or 304.

Common Errors

Examples error responses are provided below, along with the status code and scenario in which they occurred. A few considerations:

- Some of the examples are API-specific and therefore cannot be assumed to be universal to all Nike APIs.

- Not all HTTP responses include a body, and those are indicated below.

1. **400 - Bad Request:** Invalid JSON

```
{
    "message": "Validation Failed",
    "errors": [
        {
            "message": "Invalid request body",
            "code": "REQUEST_INVALID"
        }
    ]
}
```

Notes: Fix the syntax of the JSON request and retry.

1. **400 - Bad Request:** Required fields missing

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "/fieldName",
        "code": "MISSING_REQUIRED",
        "message": "Required field"
    }]
}
```

Notes: Add the mentioned required fields to the JSON request and retry.

1. **400 - Bad Request:** Field data invalid

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "/email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

Notes: Correct the mentioned field data and retry.

1. **401 - Unauthorized:** User not authenticated

```
{
    "error_id": "1ea29a72-52c1-46f3-83b3-2c93684fa4c9",
    "errors": [
        {
            "code": 35,
            "message": "Unauthorized user access"
        }
    ]
}
```

Notes: check that a valid access token was sent in the `Authorization` header. Correct and retry.

1. **403 - Forbidden:** User authenticated, but operation not allowed

```
{
    "httpStatus": 403,
    "code": "77773",
    "timestamp": 1508186768127,
    "service": "paymentapproval",
    "message": "Authorization failure",
    "errors": []
}
```

Notes: check JWT token (if applicable) and retry.

1. **404 - Not Found:** Resource does not exist

```
{
    "error_id": "3c63d8ff-bb08-4331-b89c-0bc6b4695d9a",
    "errors": [
        {
            "code": 310,
            "message": "The resource you requested does not exist."
        }
    ]
}
```

1. **405 - Method Not Allowed:**

   (no response body sent for this error)

   Notes: check API documentation for supported methods

1. **406 - Not Acceptable:**

   (no response body sent for this error)

   Notes: check `Accept` header is present and set to 'application/json' and retry.

1. **409 - Conflict:** Operation failed

```
{
    "message": "Request conflicts with previous request for same checkout",
    "code": "REQUEST_INVALID"
}
```

   Notes: retry with a never-used ID in the request (if applicable).

1. **500 - Internal Server Error:** Service had an exception

```
{
    "httpStatus": 500,
    "timestamp": 1500926568225,
    "service": "cartreviews",
    "message": "Server error"
}
```

1. **503 - Service Unavailable:** Service is down

   (no response available)

Which JSON Field Had The Error?

Nike uses the JSON Pointer standard to indicate which field of the request JSON had the error.

For example, in the error response from the Carts API you can see the field indicated as `/request/items/0/contactInfo/email`:

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "/request/items/0/contactInfo/email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

The field names are separated by `/` to indicate nesting in the structure of the request JSON.

> **TIP:** Some Nike APIs use dot notation instead of JSON Pointer, due to being built before Nike switched to the JSON Pointer standard. Check the Developer Guide for the API to confirm the error format.

Retries

Depending on the returned HTTP status code, retrying an operation might make sense or not. In general, a 400-class status means a client-side issue, while a 500-class status means a server-side issue.

Here are some recommendations for retries:

Table 9b: HTTP Error Codes with Retry Recommendations

HTTP Code	Description	Retry?	Comments
400	Bad Request	Yes - after changes	Change JSON based on response, retry operation
401	Unauthorized	Yes - after login	Once consumer logs in, retry operation
403	Forbidden	Yes - after changes	Once correct permissions of logged-in consumer are granted, retry operation
404	Not Found	No	
405	Method Not Allowed	No	Change app to call API as per spec
406	Not Acceptable	No	Change app to call API as per spec
409	Conflict	No	JSON input and query params are ok but a business rule denied the operation. No need to retry
500	Internal Server Error	No	Server issue. No need to retry until issue is resolved
503	Service Unavailable	Yes	API may come back up. Retry a few times using circuit breaker pattern

> **NOTE**: The JSON Schemas found in the API.md file should enumerate many of the possible error messages you could receive in responses. However, JSON Schemas do not include all possible error codes or messages. Specific errors can be added by each domain and are not part of these schemas.

Data Reference

User Types

Nike APIs support 3 distinct user types for commerce applications. In this guide, we will define them and discuss some ways that it can affect how you interact with the APIs.

Member

Nike's members have previously registered a Nike account and have logged in with their credentials from inside your app. Members get benefits like free shipping, free 30-day trials, and the ability to save shipping and payment information for faster checkout. For API calls involving members, an *access token* must be obtained from accounts.nike.com or Nike Unite/Identity and included in the `Authorization` request header after the user has logged in. Once Nike has validated the access token, the APIs will automatically adjust behavior as necessary based on the knowledge that the user is a member and based on Nike business rules.

Guest

The guest user has not logged in with their Nike account credentials, effectively making them a new, anonymous user to Nike. When making a purchase, the guest user must input all of their information from scratch and does not receive the additional benefits that a member would. For API calls involving guests, the `nike-visitor-id` and `appId` headers must be included with the request. The `nike-visitor-id` header value is a UUID that you get by calling Nike Unite's getVisitData function in their SDK. The `appId` header value is the identifier for your app.

Employee

The third type of user is an employee of Nike or one of it's subsidiaries (or an immediate family member of said employee) who has logged in with their Swoosh account credentials. The employee user receives special pricing on most products and may be offered different shipping options than a member or guest. For API calls involving employees, the same `Authorization` request header is used like for members.

Testing

Testing Prerequisites

The same prerequisites for interacting with APIs in the production environment also apply to test environments, i.e. registration, authorization, and JWT. Note that tokens are often unique to an environment, so your production tokens will not work in test and vice versa.

See the Prerequisites section for more info.

Test Environments

The ability to test in an environment other than production varies per API. For many APIs, functional test environments exist and should be used. For others that do not, testing in production may be an option. Contact the Product Owner of the API (listed in each Developer API Guide, At-a-Glance section) for recommendations on the best environment to use for testing.

Testing Tips

When testing isolated API calls from your local machine, here are a few tips:

- Use the Nike Developer Portal 'TRY IT NOW' feature to initiate calls from within a browser.
- Use the Postman REST client or a similar tool to test sending HTTP requests.
- Locate a small set of production skuId's to use. Reuse them whenever possible.
- For Checkout API's, use the same set of line item 'id' values for all requests. They only need to be unique with a particular request.
- Use an online UUID generator to quickly generate UUID's.
- To save time, modify the sample requests in the API guide rather than building your own.

Troubleshooting

Stuck on something with a particular API? See the below troubleshooting tips.

Query Logs With a Trace ID

If you get unexpected or confusing responses from an API, use the Trace ID from the response header to query the Nike CDT Splunk logs to get more information.

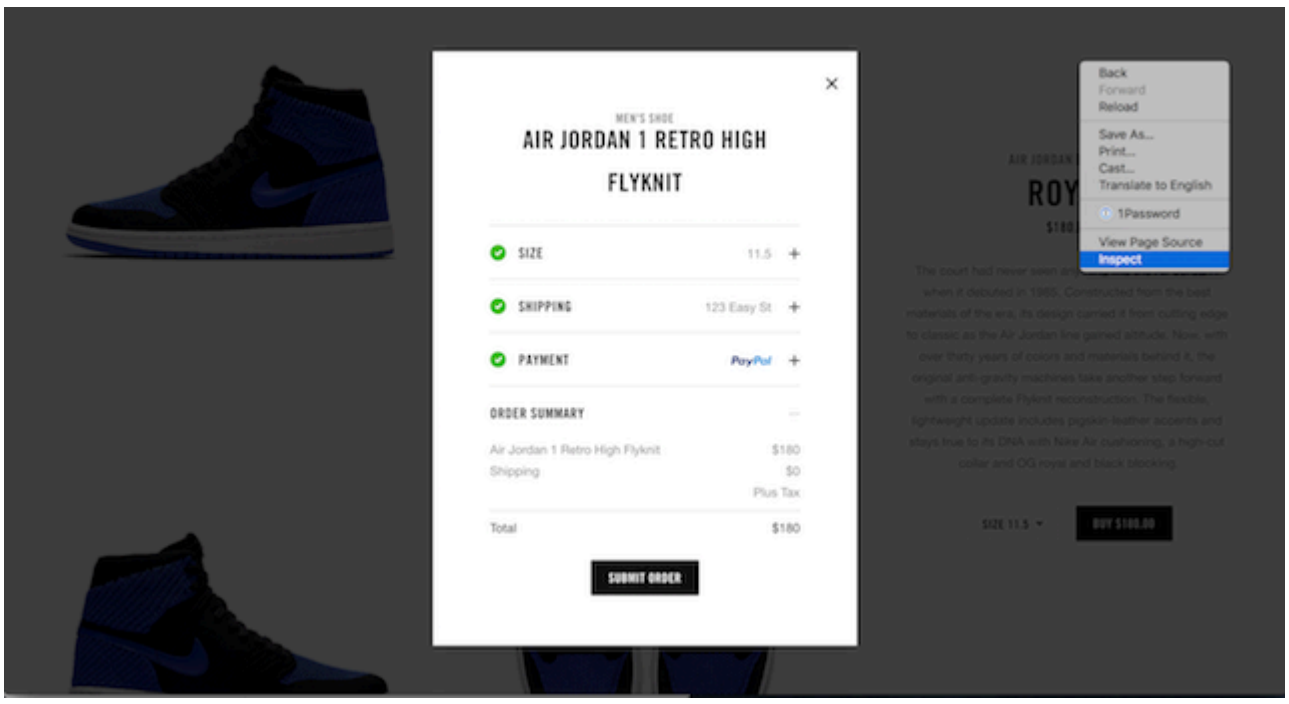
1. Find the `X-B3-TraceId` response header, which contains the Trace ID for the request, e.g. b2490b12cd639e7c. Copy the value to your clipboard.
2. Navigate to Splunk search page.
3. Paste in the Trace ID and execute the query. Once results start coming in, look for error messages or other clues as to what happened with your request.

   After looking at Splunk, if you still need assistance with troubleshooting a particular request, post a question on the relevant Slack channel (found in the Developer API Guide) and be sure to include your Trace ID.

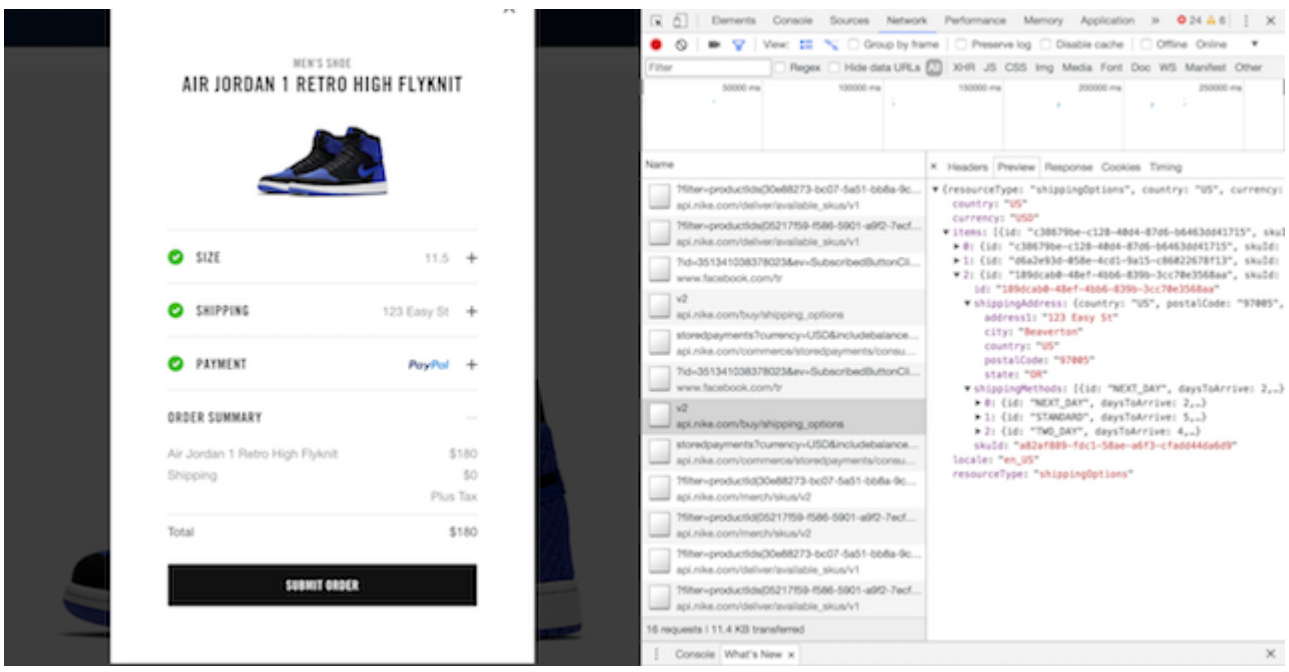
Inspect Browser Activity in a Live Experience

Use your browser's built-in tools for inspecting the web service calls which occur for a live Nike experience like SNKRS Web. Sometimes seeing what other experiences are doing might address your question or concern. For example, to follow the order of calls made when changing a shipping address during checkout:

1. Right-click anywhere in browser main window, select `Inspect`.



1. In Inspect window, select `Network` tab.
2. Perform the action in the main window. In this case, change the shipping address and click ‘Save and Continue’.
3. In the `Network` tab, scan through the list for any items with “api.nike.com”. In this case, click to select the call to “api.nike.com/buy/shipping_options”.



1. Study the data in the Headers, Preview, and Response tabs. Is there some request header data present that you hadn’t considered? Is the data in the request body or response body as expected?

Circuit Breaker Best Practices

Be a good client by following these Circuit Breaker Best Practices when calling Nike APIs.

Document Change Log

Summary	Date
Initial publish	10/18/2018
Added making Your First Request and other edits	12/11/2018
Added Authentication router	08/20/2019
Added Consumer and S2S JWT to Authentication section	09/24/2019
Updated with UER, OSCAR, OIDC	05/11/2022
Added accounts.nike.com	01/04/2023

Related Links

- API Standards
- Glossary

- Circuit Breaker Best Practices